

mecaniQA – Aracaju

Autor: João Victor Carvalho de Oliveira
Kaio Santo de Oliveira
Marco Antonio Almeida Silva
Nivton Amaral Alves
Robert de Araújo Costa

Agenda

O objetivo desta apresentação é apresentar a Web API desenvolvida para a mecaniQA, destacando sua modelagem, funcionalidades e principais conceitos utilizados.

1. Contexto e arquitetura

Estrutura geral da API e fluxo das requisições

2. Modelagem das classes

Entidades, enumeração, controllers e repositories.

3. CRUD e rotas REST

Operações de cadastro, consulta, atualização e exclusão.

4. Singleton e persistência

Armazenamento em memória e gerenciamento das instâncias

5. Validação e tratamento de erros

Validação dos dados e utilização dos status HTTP

Contexto e Arquitetura da API



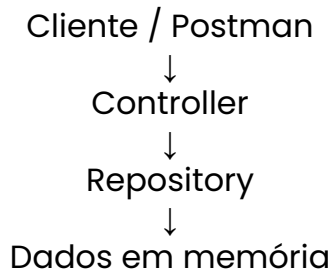
Objetivo do projeto

Desenvolver uma Web API REST
para gerenciamento de Peças e
Serviços da mecaniQA

Tecnologias

Java 21
Spring Boot
Gradle

Arquitetura



```
16 @RestController
17 @RequestMapping("/api/pecas")
18 public class PecaController {
19
20     private final PecaRepository repository = PecaRepository.getInstance();
21
22     // ===== CREATE =====
23     @PostMapping
24     public ResponseEntity<Peca> salvar(@Valid @RequestBody Peca peca) {
25         Peca pecaSalva = repository.salvar(peca);
26         return ResponseEntity.status(HttpStatus.CREATED).body(pecaSalva);
27     }
```



Gradle



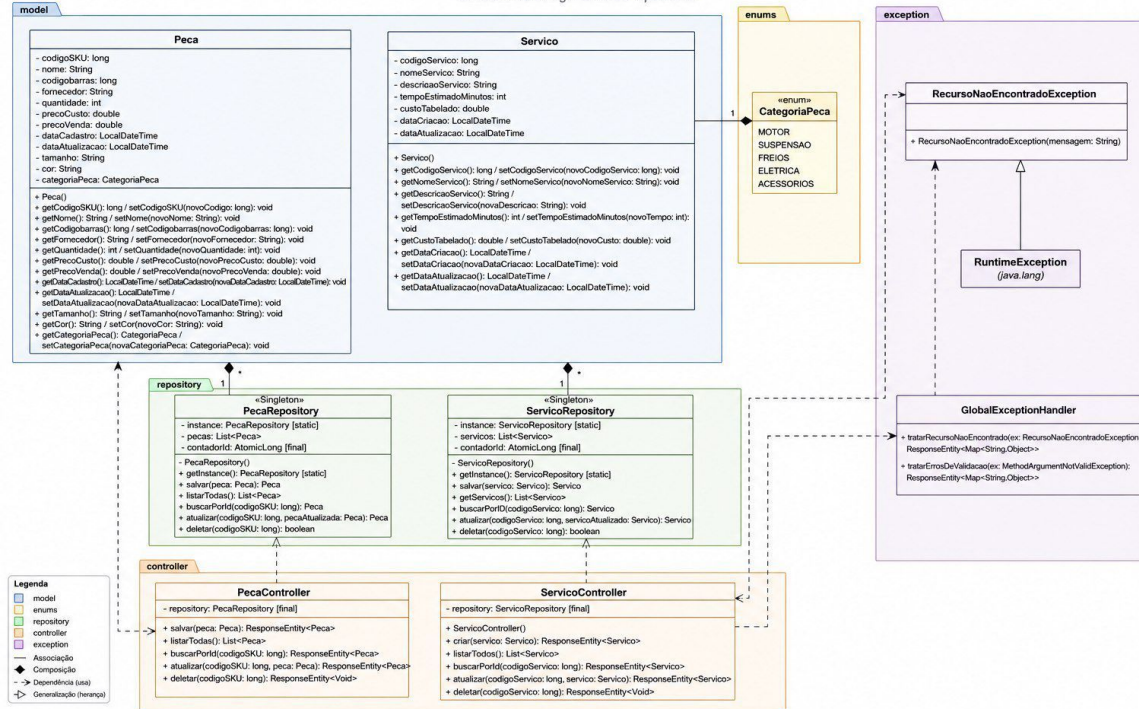
Java 21

mecaniQA – Aracaju

Modelagem de classes

Diagrama de Classes — mecaniQA API

Baseado no código-fonte do repositório



CRUD e Rotas REST



Peças

POST /api/pecas
Criar peça

GET /api/pecas
Listar peças

GET /api/pecas/{id}
Buscar peça

PUT /api/pecas/{id}
Atualizar peça

DELETE /api/pecas/{id}
Excluir peça

Serviços

POST /api/servicos
Criar serviço

GET /api/servicos
Listar serviços

GET /api/servicos/{id}
Buscar serviço

PUT /api/servicos/{id}
Atualizar serviço

DELETE /api/servicos/{id}
Excluir serviço

Status HTTP

201 Created
Cadastro realizado

200 OK
Consulta / atualização

204 No Content
Exclusão realizada

404 Not Found
Recurso não encontrado

400 Bad Request
Dados inválidos

Singleton, Validação e Tratamento de Erros



Singleton

- Construtor privado
- Instância única
- Método getInstance()
- Dados mantidos em memória

```
16 @RestController
17 @RequestMapping("/api/pecas")
18 public class PecaController {
19
20     private final PecaRepository repository = PecaRepository.getInstance();
21
22     // ===== CREATE =====
23     @PostMapping
24     public ResponseEntity<Peca> salvar(@Valid @RequestBody Peca peca) {
25         Peca pecaSalva = repository.salvar(peca);
26         return ResponseEntity.status(HttpStatus.CREATED).body(pecaSalva);
27     }
```

Validação

- @Valid
- @NotBlank
- @NotNull
- @Positive
- @PositiveOrZero

Validação dos dados recebidos nas requisições.

mecaniQA – Aracaju

Tratamento de erros

- 400 Bad Request
→ Dados inválidos
- 404 Not Found
→ Recurso não encontrado

```
13 @RestControllerAdvice
14 public class GlobalExceptionHandler {
15
16     @ExceptionHandler(RecursoNaoEncontradoException.class)
17     public ResponseEntity<Map<String, Object>> tratarRecursoNaoEncontrado(
18         RecursoNaoEncontradoException ex) {
19         Map<String, Object> corpo = Map.of(
20             "status", HttpStatus.NOT_FOUND.value(),
21             "erro", "Recurso não encontrado",
22             "mensagem", ex.getMessage(),
23             "timestamp", LocalDateTime.now().toString()
24         );
25         return ResponseEntity.status(HttpStatus.NOT_FOUND).body(corpo);
26     }
```

Considerações finais



Principais resultados

- Implementação do CRUD de Peças e Serviços
- Criação de rotas REST
- Modelagem das entidades utilizando classes e enumeração
- Implementação do Padrão Singleton
- Persistência dos dados em memória
- Validação das requisições
- Tratamento de exceções
- Utilização adequada dos status HTTP

O projeto estabelece a base da API da mecaniQA para as próximas etapas do desenvolvimento.

Referências



MECANIQA AUTOMOTIVE TECH.

OAT 1 — Introdução a Diagrama de
Classes. Programa de Trainee
2026.2.

GitHub.

mecaniQA-api-aracaju. Repositório
do projeto da equipe.

[github.com/JottavDEV/mecani
QA-api-aracaju](https://github.com/JottavDEV/mecaniQA-api-aracaju)